The Differential

East bay 1: bevel gears that add and subtract · 14 November 2026

Gears that add. Two side gears, A and B, face each other on one axis. Two spider gears between them turn on a cross pin in the carrier, the ring round the middle. Whatever you do with A and B:

```
carrier = (A + B) / 2
```

The inputs' drums count ten units a turn. The carrier's ring has twenty numbers round it, so it reads **A + B**. Turn an input backwards and it subtracts.

The same mechanism sits in the back axle of a car, and dozens of them summed the quantities in the fire-control computers of the Second World War. It is the first building block of the hall's analog machines.

Open `differential.lua` in bpp, or walk to it in Babbage's Hall. It starts with the tour: - 3 + 4 = 7 - add 5 to B: 3 + 9 = 12 - take 2 from A: 1 + 9 = 10 - back to 0 + 0

Keys

Key	
`1` / `Q`	input A: one unit forward / back
`2` / `W`	input B: one unit forward / back
`0`	both back to 0
`T`	the tour

What to watch

- **Turn A alone:** the spiders roll round the held B and take the ring half as far.
- **Turn both the same way:** the whole cluster turns as one, and the spiders don't spin.
- **Turn them opposite ways:** the spiders spin and the ring stays still.

Each counter is read at the front, where its brass pointer points. The pointers stand just outside the numbers, the drums' on their outer sides, so nothing that turns can touch them.

Building it

Run `./make-stls.sh` (needs OpenSCAD). The parts come out in `stl/`, each turned to print without supports.

Part	How many	
`side.stl`	2	a side gear (16 teeth, 60 mm) with its hub; printed teeth down
`spider.stl`	2	the spider gears
`carrier-half.stl`	2	the carrier, in two halves glued together at the ring's middle
`drum.stl`	2	the input counters, 0 to 9 (paint the raised numbers)

Part	How many	
`crank.stl`	2	mount them mirror images: both handles point outward
`frame.stl`	1	base, posts and pointers

You also need: - **two 5 mm steel rods,** about 75 mm each: the inputs (from inside a side gear out through its crank). - **one 5 mm rod,** about 100 mm: the cross pin the spiders turn on. - **glue,** and a little grease for the teeth.

Assembly: 1. Glue each side gear on its rod. 2. Thread the carrier's halves over the hubs, put the spiders on the cross pin between them, and glue the halves. 3. Slide the drums onto the rods, and the rods through the posts. 4. Glue the drums and cranks to the rods with both drums and the ring showing 0 at the pointers, each crank's handle pointing outward.

Print settings: PLA, 0.2 mm layers, 50% infill. The simulation's masses assume about 0.8 g/cm³.

The gears are GregFrost's parametric involute bevel gears (GPL), in `parts/gearsv50.scad`, brought up to date for current OpenSCAD (its `assign()` and `polyhedron(triangles=)` are gone from newer versions, and the teeth with them; the gears come out exactly as before).

The simulation

Units are centimetres, kilograms and seconds.

- **Gears:** the four gears are the printed ones, and they turn each other by their teeth alone.
- In the physics, each gear is a compound of convex pieces: its body, and its sixteen teeth, cut from the same OpenSCAD shapes by `parts/make-hulls.py`. That script also works out each part's mass and moment of inertia from the printed shape.
- Bullet's mesh-to-mesh contacts jammed and slipped on these teeth; convex pieces work. See `FINDINGS.md`.
- **The hand:** the inputs are turned by a hand (a motor steered to where the keys put them), one input at a time, holding the other where it was set. The carrier and spiders move only as the teeth push them.
- **Friction and steps:** tooth friction 0.3 (a slider), 2400 steps a second.
- **Left out:** the carrier's bearing on the hubs, and the rods' bearings in the posts.

Headless testing: `bpp -f differential.lua -n 2400` runs the tour and prints `RESULT differential tour: PASS` or `FAIL`. `DF_A`, `DF_B` and `DF_TOUR=0` turn the inputs to given values; `DF_LOG=1` prints the readings each second. Once the hand is done and the gears are still, it rests (stops stepping) until there's something to do; `DF_REST=0` never rests, `DF_REST_LOG=1` shows when it does.